ApproveHub

Digital Document Approval System

Flow Blueprint with an equal workload split: from first install to final submission

Item	Details
Course	EDI1 (ES26102), F.Y. B.Tech IT, VIT Pune
Team leader	Shreeyash Solunke
Team	Shreeyash, Shravani, Shreya Sutar, Shreya Loya
Technology	Python, Flask, SQLite, SQLAlchemy, HTML, CSS, JavaScript, Flutter, SHA-256, PyMuPDF
Plan	8 weeks, 38 steps, 6 phases (Phase 0 to Phase 5)
Workload	Equal split of 195 effort points: Shreeyash 51, Shravani 48, Shreya Sutar 47, Shreya Loya 49. The leader owns all 8 critical steps.

The flow of this document

Section	What it shows	Read it to
1	How to read this blueprint	Understand colours, arrows and step labels
2	Master roadmap	See the whole project as 6 connected phases
3	How the system works	Follow a document from upload to final PDF
4	Words you will see	Learn the terms in simple language
5	Equal workload split and dependencies	See every member's tasks, effort points and who waits for whom
6	Team swimlane flow	See every step by person and in order
7	Phase-by-phase steps	Build: inputs, steps in order, outputs, detailed cards
8	Learning flow	Know what to study each week
9	Tools, topics and technologies	Detailed learning tables with practice tasks
10	API list and tools to install	Set up and plan the endpoints
11	Weekly plan, rules, checklist	Track progress and finish properly

1. How to read this blueprint

Two minutes now saves hours later.

- **Direction:** the document is built as one flow. Master roadmap (big picture) then system flow, team swimlane, and finally each phase in detail.
- **Colours:** dark blue = Shreeyash, green = Shravani, purple = Shreya Sutar, orange = Shreya Loya, grey = everyone.
- **Arrows:** an arrow from one box to another means 'do this first, then that'. Boxes on the same level with no arrow between them can be built at the same time.
- **Phases:** Phase 0 to Phase 5. Never start a phase's main building before the previous phase's outputs exist.
- **Phase page layout:** every phase starts with **Inputs** (what must already exist), **Steps in order**, **Parallel work** and **Outputs** (what you will have at the end). Then come the detailed step cards.
- **Step card fields:** Why it matters, What to do, How (numbered actions), Done when (your check), Learn first, Needs and Unlocks (which steps come before and after), and Importance (Critical, High or Medium) with Effort in points.
- If stuck for more than 45 minutes, ask a teammate. Then ask your guide or read the official documentation.

Golden rule: build a small working version of the whole flow first (upload, pick approvers, approve, stamp). Add extra features only after that flow works end to end.

2. Master roadmap

The whole project as 6 connected phases. Each phase feeds the next.

PHASE 0
Week 1

Setup and planning

Tools, repository, database design, API list, UI designs, dummy data

↓

PHASE 1
Weeks 2-3

Foundation

Database in code, register and login, roles, upload with SHA-256, web and app login, profile API

↓

PHASE 2
Weeks 4-5

Core approval workflow

Approver chain, sequence logic, approve / pending / compliance, timeline, hash checks, dashboards

↓

PHASE 3
Week 6

Document processing and extras

Stamping signatures and seals, final hash, PIN lock, status and download pages, admin panel

↓

PHASE 4
Week 7

Integration, testing and deployment

Join web, app and API, test everything, fix bugs, put the project online

↓

PHASE 5
Week 8

Documentation and submission

README, report, PPT, demo rehearsal, final review and submission

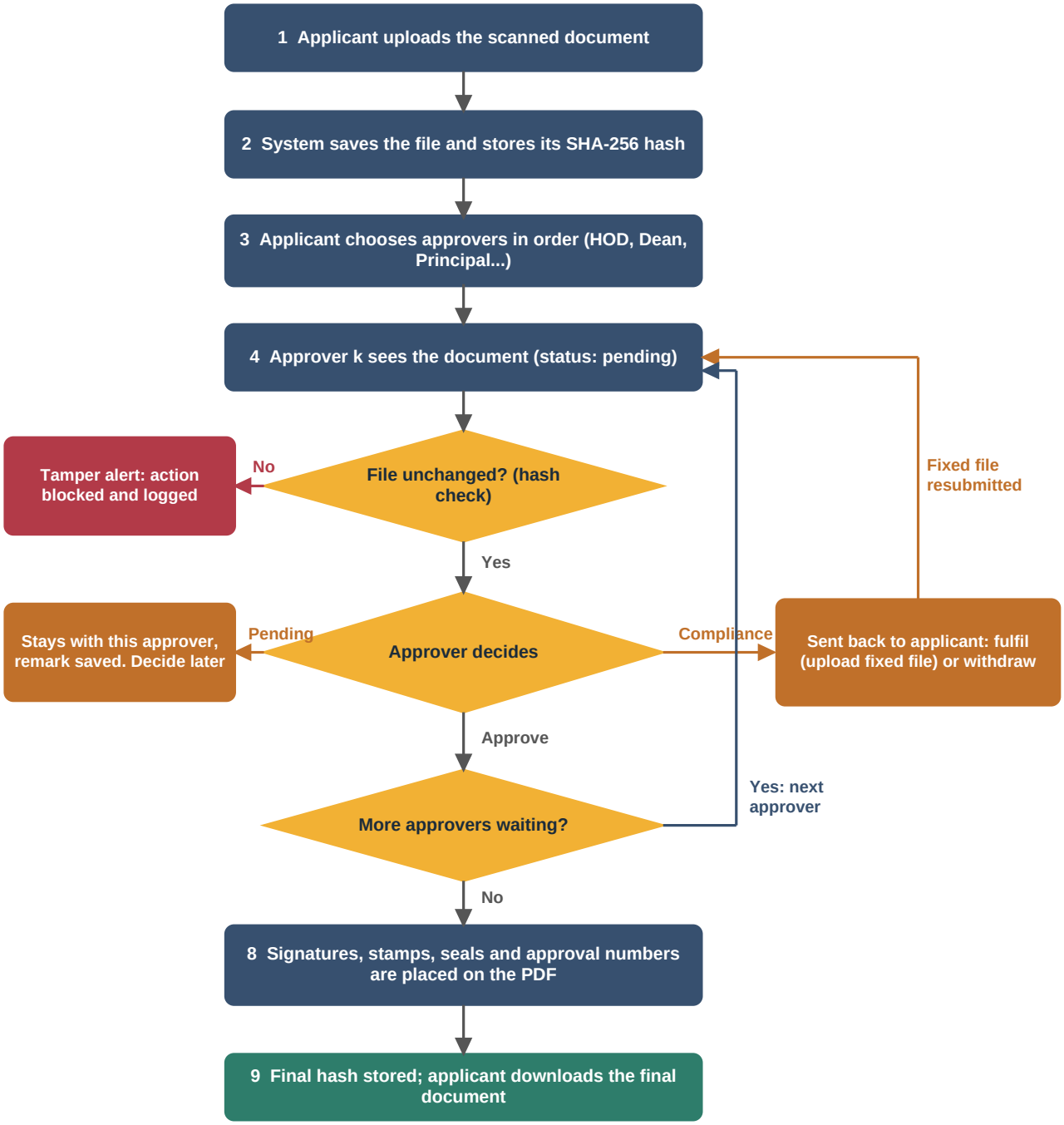
Read it top to bottom. The outputs of one phase are the inputs of the next. Phase 2 is the heart of the project (the approval workflow), and Phase 3 produces the final stamped document.

3. How the system works

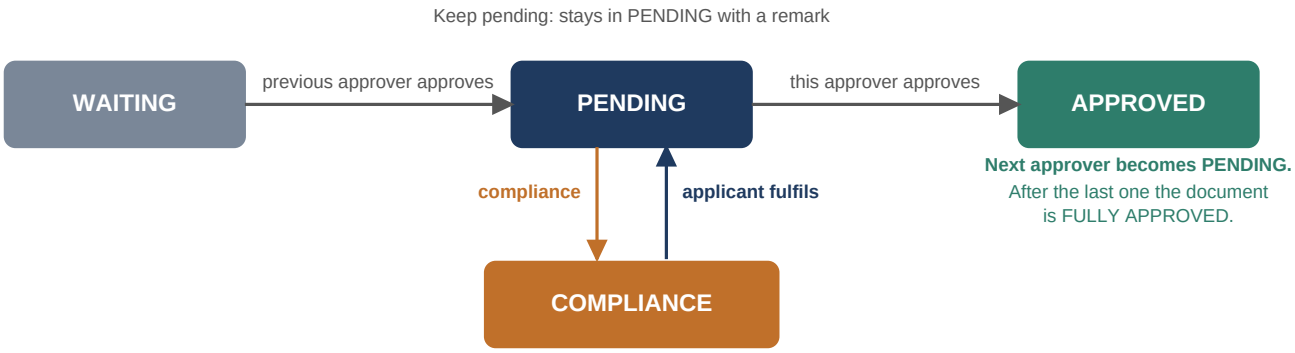
Follow one document from upload to the final stamped PDF.

ApproveHub is a website and mobile app where a student or staff member uploads a scanned document (for example a leave letter), picks the authorities who must approve it, and the document travels from one authority to the next. Each authority can **approve**, **keep pending** or **send compliance**. When everyone has approved, signatures, stamps, seals and approval numbers are placed on the document. If anyone secretly edits the file, the system detects it.

3.1 Document journey (flowchart)



3.2 Status of one approval step



3.3 How the parts connect

Website HTML, CSS, JavaScript (Shravani)	**Mobile app** Flutter (Shreya Sutar)
API calls (JSON over HTTP)	
Backend: Flask API Login and roles, sequence logic, actions, timeline, hashing, document processing (Shreeyash) Profile, admin, search APIs (Shreya Loya)	
Database (SQLite) users, documents, approval_steps, timeline	**File storage** uploads, signatures, stamps, final PDFs

4. Words you will see

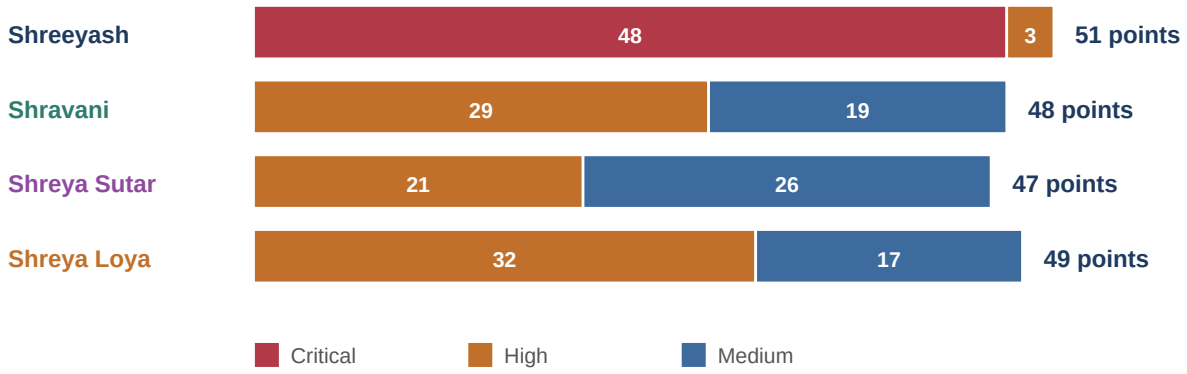
Simple meanings of the technical words.

Word	Simple meaning
Backend	The hidden part that stores data and applies the rules (Python and Flask)
Frontend	What the user sees and taps (website and app)
API / endpoint	A web address the frontend calls to ask the backend to do something, for example POST /api/documents/upload
Database	Where data is stored permanently (SQLite). Tables are like Excel sheets
Model	A Python class that describes one table, for example User or Document
ORM (SQLAlchemy)	A helper that lets you work with database tables using Python code instead of SQL
Hash (SHA-256)	A 64-character fingerprint of a file. Change one byte and the fingerprint changes completely
Password hashing	Storing a scrambled form of the password so even you cannot read it
Session / JWT	How the system remembers that a user is logged in
Role	What a user is allowed to do: applicant, approver or admin
State machine	A document is always in exactly one state (waiting, pending, approved, compliance) and moves by fixed rules
Git / GitHub	Tool and website to share code and track changes among the four of you
Branch	Your own copy of the code to work safely, merged into main when ready
JSON	The text format backend and frontend use to exchange data
Postman	A tool to send requests to your API and see the reply, without any screen
Flutter / Dart	Tool and language used to build the mobile app
Compliance	An approver asking the applicant to fix or add something before approval
Deployment	Putting the project on an online server so anyone can open it

5. Equal workload split and dependencies

Every member gets about the same effort. The leader's share is made of the most critical tasks.

Effort points are estimates for a beginner: **1 point is about 1 hour** of focused work. There are 34 individual tasks worth 195 points in total, so a fair share is about 49 points each. Steps 1, 3, 33 and 38 are done by everyone for their own module and count equally for all four.



How the balance works: the numbers inside the bars are effort points by importance. Shreeyash owns **all 8 Critical steps** (database, SHA-256, approval sequence, actions, hash check, stamping, integration). The others own High and Medium steps that are just as much work, but with less risk to the project.

Summary of the split

Member	Role in the project	Effort	Critical / High / Medium
Shreeyash (leader)	Core engine: database, tamper detection (SHA-256), the approval workflow, document stamping and final integration.	51 points	48 / 3 / 0
Shravani	Access, web and going online: login and roles, uploads, search, all website pages and deployment.	48 points	0 / 29 / 19
Shreya Sutar	Design and mobile: UI designs, the Flutter app, and the small APIs the app needs (timeline, PIN, final download).	47 points	0 / 21 / 26
Shreya Loya	Workflow support and quality: API list, approver chain, compliance, profile and admin APIs, testing, report and PPT.	49 points	0 / 32 / 17

Every member's tasks

Shreeyash - 51 points

Core engine: database, tamper detection (SHA-256), the approval workflow, document stamping and final integration.

Step	Task	Importance	Effort	Week
2	Create the project and the shared GitHub repository	High	3 pts	1
4	Design the database tables	Critical	5 pts	1
8	Create the database in code	Critical	5 pts	2
12	SHA-256 fingerprint at upload	Critical	5 pts	3
17	Multi-approver sequence logic	Critical	8 pts	4
18	Approve, pending and compliance actions	Critical	7 pts	4
21	Hash verification before each action	Critical	4 pts	5
25	Stamp signatures, stamps, seals and approval number	Critical	9 pts	6
31	Integrate web and app with the real API	Critical	5 pts	7

Shravani - 48 points

Access, web and going online: login and roles, uploads, search, all website pages and deployment.

Step	Task	Importance	Effort	Week
7	Prepare dummy data and sample files	Medium	3 pts	1
9	Registration and login with password hashing	High	6 pts	2
10	Role-based access	High	4 pts	2
11	Document upload with validation	High	5 pts	3
13	Base layout, login and registration pages	Medium	5 pts	2
22	Applicant and approver dashboards (web)	High	9 pts	4
24	Search and filter API	Medium	4 pts	5
28	Status page, search and filter UI, download page (web)	Medium	7 pts	6
34	Deploy online	High	5 pts	7

Shreya Sutar - 47 points

Design and mobile: UI designs, the Flutter app, and the small APIs the app needs (timeline, PIN, final download).

Step	Task	Importance	Effort	Week
6	Create UI designs for all screens	Medium	8 pts	1
14	Flutter project and login screen	Medium	6 pts	2
20	Live approval timeline	High	4 pts	5
23	Applicant and approver screens (app)	High	9 pts	4
26	Final document and final hash	High	4 pts	6
27	PIN lock logic	High	4 pts	6
29	Timeline, profile, PIN and biometric lock (app)	Medium	8 pts	6
35	README and API documentation	Medium	4 pts	8

Shreya Loya - 49 points

Workflow support and quality: API list, approver chain, compliance, profile and admin APIs, testing, report and PPT.

Step	Task	Importance	Effort	Week
5	Write the API list	Medium	4 pts	1
15	Approver profile API (signature, stamp, seal)	High	5 pts	3
16	Approver selection and approval chain	High	6 pts	4
19	Compliance flow (take back or fulfil)	High	6 pts	5
30	Admin panel and download center API	High	7 pts	6
32	Full testing and bug list	High	8 pts	7
36	Final report, literature survey and references	Medium	8 pts	8
37	Presentation and demo script	Medium	5 pts	8

Done by everyone together

Step	Task	What each person does
1	Install your tools	Installs the tools for their own role
3	Learn your week 1 basics	Studies the basics for their own module
33	Fix bugs and run a security check	Fixes the bugs found in their own module
38	Final review and submission	Presents their own module and helps with the final fixes

Why this split is fair and still puts the most important work with the leader

- **Equal effort:** the four totals are within a few points of each other (see the chart).
- **Most important with the leader:** a failure in the database, hashing, approval sequence, actions or stamping breaks the whole project, so these stay with Shreeyash. He also integrates everything at the end.
- **Each person owns a clear module:** Shravani = access, web and deployment. Shreya Sutar = design, mobile and the app's small APIs. Shreya Loya = workflow support, admin and quality.
- **Review and help:** Shreeyash reviews every merge on the critical modules and helps teammates when they are stuck. Teammates ask early rather than waiting.
- **Learning curve:** weeks 1 to 3 are lighter on purpose, because everyone is learning. The heavier steps come in weeks 4 to 7.

Key dependencies (who waits for whom)

Dependency	Provided by	Needed by	Needed by when
Database tables and field names (Step 4)	Shreeyash	Everyone	End of Week 1
API list with exact fields (Step 5)	Shreya Loya, reviewed by Shreeyash	Shravani, Shreya Sutar, Shreeyash	End of Week 1
UI designs (Step 6)	Shreya Sutar	Shravani and Shreya Sutar	Week 2
Dummy data and sample files (Step 7)	Shravani	Everyone testing	Week 2
Login and token API (Step 9)	Shravani	Web pages, app screens, every protected endpoint	Week 2
Upload API (Step 11) and SHA-256 (Step 12)	Shravani, then Shreeyash	Approver chain and actions	Week 3
Profile API and image format (Step 15)	Shreya Loya	Shreeyash for stamping (Step 25)	Week 3 (agree early)
Approver chain (Step 16)	Shreya Loya	Shreeyash for sequence logic (Step 17)	Week 4
Approve, pending, compliance (Step 18)	Shreeyash	Compliance flow (Step 19), timeline (Step 20), dashboards (Steps 22, 23)	Weeks 4-5
Stamped final document (Step 25)	Shreeyash	Final download (Step 26)	Week 6
Timeline and download endpoints (Steps 20, 26)	Shreya Sutar	Web status and download pages (Step 28), app timeline (Step 29)	Week 6
All modules finished	Everyone	Shreeyash for integration (Step 31)	Week 7

Shared by all four

- Write the report section for your own module.
- Hold a 15-minute weekly meeting. Each person answers: what I finished, what I will do next, what is blocking me.
- Use one GitHub repository. Work on your own branch. Tell the owner before editing someone else's module.
- Test your own module before handing it over.

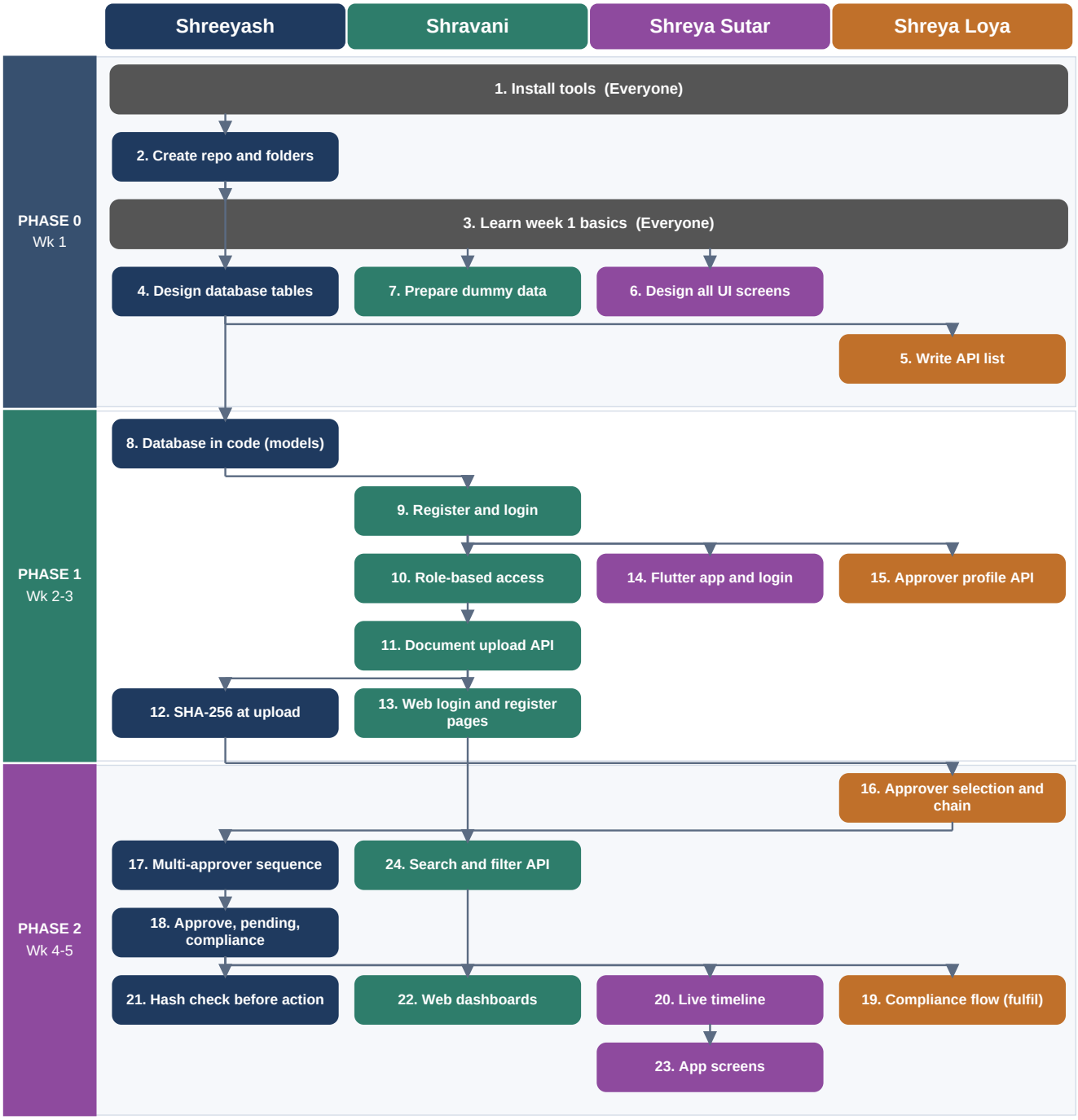
Career value of each share

Member	Skills gained	Roles it helps for
Shreeyash	Database design, SHA-256 and data integrity, workflow and state logic, PDF processing, system integration, code review	Backend, security and full-stack internships
Shravani	Authentication and role-based access, Flask APIs, file uploads, full web frontend, deployment	Full-stack and web developer roles
Shreya Sutar	UI/UX design, Flutter and Dart, small Flask APIs, technical documentation	Mobile developer and product design roles
Shreya Loya	Relational data and workflow features, admin systems, API documentation, QA testing, technical writing	QA, junior backend and technical writing roles

6. Team swimlane flow

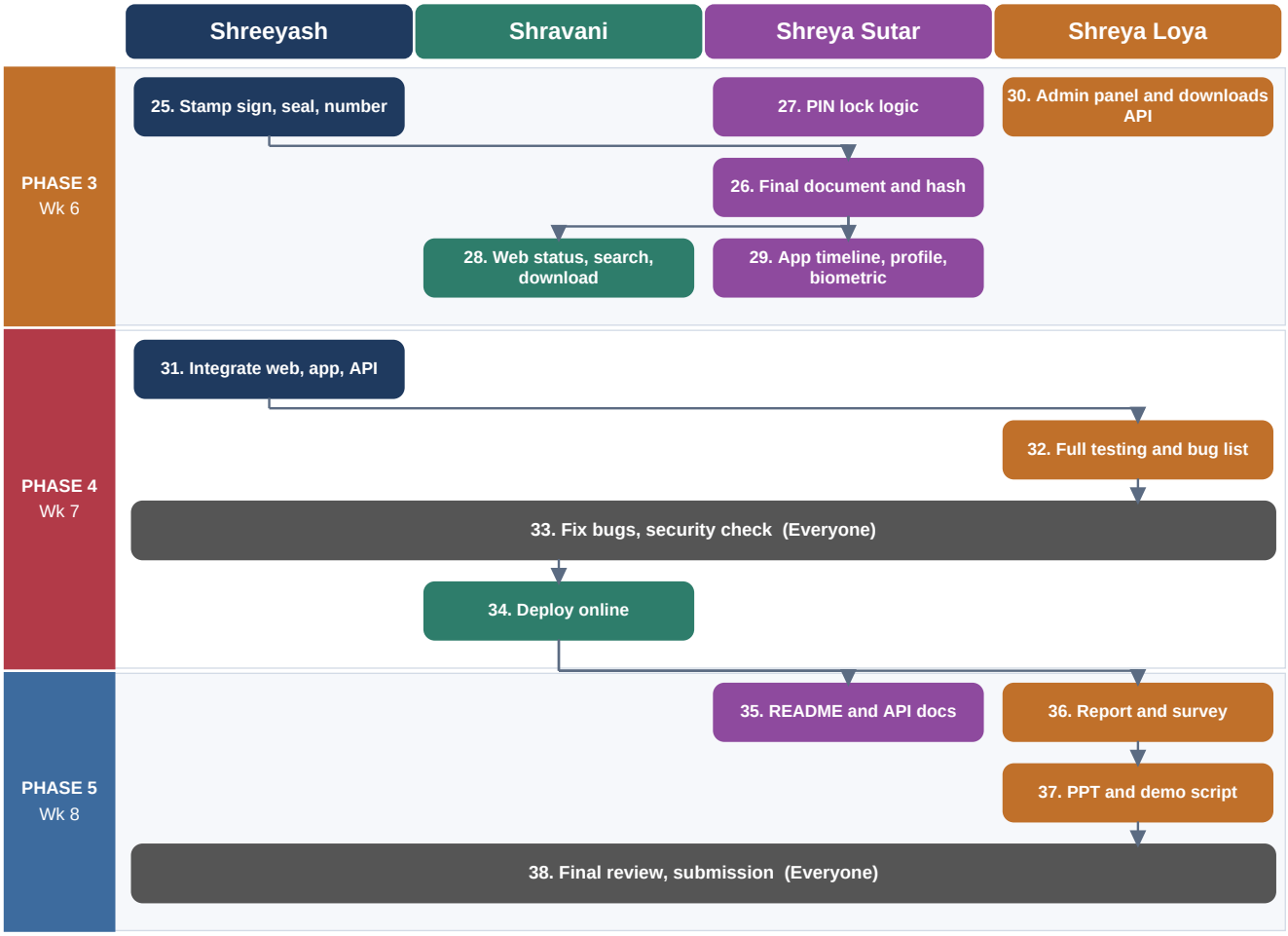
Every step by person, in the order to follow. Read each phase from top to bottom.

Phases 0 to 2



Arrows show the order to follow. Boxes on the same level can be built in parallel.

Phases 3 to 5



Arrows show the order to follow. Boxes on the same level can be built in parallel.

Where the people meet: the API list (Step 5, Shreya Loya) lets Shravani and Shreya Sutar start; the profile API (Step 15, Shreya Loya) feeds Shreeyash's stamping (Step 25); the approver chain (Step 16, Shreya Loya) feeds Shreeyash's sequence logic (Step 17); the timeline and download endpoints (Steps 20 and 26, Shreya Sutar) feed the web and app pages; Step 31 brings all four lanes together.

7. Phase-by-phase step flow

For each phase: inputs, steps in order, parallel work, outputs, then the detailed step cards.

PHASE 0 — Setup and planning (Week 1)

Goal: Everyone has tools installed, the shared repository exists, and the database and API plan are written, so nobody is blocked later.

Phase flow	Details
INPUTS (must exist before this phase)	Team formed and project topic (ApproveHub) chosen.
STEPS IN ORDER	**Step 1** - Install your tools *(Everyone)* **Step 2** - Create the project and the shared GitHub repository *(Shreeyash)* **Step 3** - Learn your week 1 basics *(Everyone)* **Step 4** - Design the database tables *(Shreeyash)* **Step 5** - Write the API list *(Shreya Loya)* **Step 6** - Create UI designs for all screens *(Shreya Sutar)* **Step 7** - Prepare dummy data and sample files *(Shravani)*
PARALLEL WORK	After Step 2: Steps 3, 4, 6 and 7 can run in parallel. Step 5 follows Step 4.
OUTPUTS (you will have)	Tools installed; GitHub repository; database design; API list; UI designs; dummy data and sample files.

STEP 1 — Everyone — Week 1

Install your tools

Why it matters: Without the same tools, code that works on one laptop fails on another.
What to do: Get your computer ready for the project.
How, one action at a time:

1. Install Python 3.11 or newer (tick 'Add Python to PATH' on Windows), VS Code, Git and Postman.
2. Shreya Sutar also installs Flutter SDK and Android Studio. Shreeyash, Shravani and Shreya Loya also install DB Browser for SQLite.
3. Open a terminal and run `python --version` and `git --version`. Both must show a version number.
4. Create a GitHub account if you do not have one. Share your GitHub username in the team group.

Done when: every member can run both commands and has a GitHub account.
Needs: Nothing, this is the starting point. **Unlocks:** Steps 2, 3.

> **Tip:** Install everything on the same day and help each other. A broken setup is the number one reason beginners lose a week.

STEP 2
Shreeyash
Week 1

Create the project and the shared GitHub repository

Why it matters: One shared repository is how four people write one project without overwriting each other.
What to do: Create the folder structure and put it on GitHub so all four of you can work on it.
How, one action at a time:

1. Create folders: backend, web, mobile, docs, uploads.
2. Create a virtual environment: `python -m venv venv`, then activate it (`venv\Scripts\activate` on Windows).
3. Install the libraries: `pip install flask flask-sqlalchemy pillow pymupdf reportlab`, then run `pip freeze > requirements.txt`.
4. Run `git init`, make a .gitignore (ignore venv and uploads), commit, create a repo on GitHub and push.
5. Invite the 3 members. Teach them to clone, create their own branch (`git checkout -b shravani-web`) and push it.

Done when: all three teammates have cloned the repo and pushed one small test file on their own branch.
Learn first: Git basics: init, add, commit, branch, push, pull, merge.
Needs: Step 1. **Unlocks:** Steps 4, 7.
Importance: High | **Effort:** 3 points (about 3 hours)

> **Tip:** Rule for the team: nobody pushes directly to main. Work on your branch, then ask Shreeyash to merge.

STEP 3
Everyone
Week 1

Learn your week 1 basics

Why it matters: You will not understand the later steps without these basics.
What to do: Study just enough to start building. Do not try to learn everything at once.
How, one action at a time:

1. Everyone starts with Git and how HTTP works. Then: Shreeyash: Python basics, SQL tables and project structure. Shravani: Python and Flask basics, HTML and CSS. Shreya Sutar: UI/UX basics, Figma and Dart basics. Shreya Loya: Python and Flask basics, REST and API design basics.
2. Everyone: how HTTP requests, JSON and APIs work. Watch one short video, then send a request to any public API using Postman.
3. Everyone: Git commands from Step 2.

Done when: each person has built one tiny practice program in their own area (see Section 9 for practice tasks).
Needs: Step 1. **Unlocks:** Step 6.

STEP 4
Shreeyash
Week 1

Design the database tables

Why it matters: The database is the memory of the project. Changing it later breaks everyone's code, so decide now.
What to do: Decide exactly what data the system stores before writing any code.
How, one action at a time:

1. Draw 4 tables on paper or in draw.io:
2. **users**: id, name, email, password_hash, role, post, department, signature_path, stamp_path, seal_path, pin_hash.
3. **documents**: id, title, file_path, original_hash, final_path, final_hash, applicant_id, status, current_step, created_at.
4. **approval_steps**: id, document_id, approver_id, step_order, status, remark, approval_no, acted_at.
5. **timeline**: id, document_id, event, time.
6. Mark the relationships: one user has many documents; one document has many approval steps and many timeline events.

Done when: the drawing is shared in the team group and everyone agrees field names will not change without telling the others.
Learn first: Tables, primary key, foreign key, one-to-many relationship.
Needs: Step 2. **Unlocks:** Steps 5, 8.
Importance: Critical | **Effort:** 5 points (about 5 hours)

> **Tip:** Think of each table as an Excel sheet and each row as one record. A foreign key is just a column that points to a row in another sheet.

STEP 5
Shreya Loya
Week 1

Write the API list

Why it matters: Shravani and Shreya Sutar cannot build their pages until they know which addresses to call.
What to do: List every endpoint so frontend and app builders know what to call and what they get back. Shreeyash reviews it before it is shared.
How, one action at a time:

1. Make a table with: method (GET or POST), URL, what is sent, what comes back. Example: `POST /api/login` receives email and password, returns token and role.
2. Use the starter list in Section 10 as your base and fill in the exact fields.
3. Save it in the docs folder and share it. Explain it to each teammate in 10 minutes.

Done when: Shreeyash has reviewed the list, and Shravani and Shreya Sutar confirm they understand each endpoint.
Learn first: REST basics: methods, URLs, JSON request and response.
Needs: Step 4. **Unlocks:** Steps 13, 14.
Importance: Medium | **Effort:** 4 points (about 4 hours)

STEP 6
Shreya Sutar
Week 1

Create UI designs for all screens

Why it matters: Designing first means the code is written once, not rewritten after the team changes its mind.
What to do: Design how every web page and app screen looks.
How, one action at a time:

1. Pick a colour theme (2 main colours), a logo and an icon style.
2. In Figma or Canva design: login, register, applicant dashboard, upload and approver selection, status timeline, approver dashboard, profile (signature, stamp, seal upload), PIN screen, admin panel, download center.
3. Make one version for web (wide) and one for the app (narrow).
4. Export images and share the link with the team.

Done when: all screens exist as designs and the team has agreed on them. Shravani and Shreya Sutar build from these.
Learn first: Basic UI/UX, Figma frames and components.
Needs: Step 3. **Unlocks:** Steps 13, 14.
Importance: Medium | **Effort:** 8 points (about 8 hours)

STEP 7
Shravani
Week 1

Prepare dummy data and sample files

Why it matters: Realistic test data lets everyone test from day one instead of waiting for the full system.
What to do: Create realistic test data and sample files.
How, one action at a time:

1. Make a list of 10 dummy users: 3 applicants, 6 approvers with posts like HOD, Dean, Principal, Professor, Director, and 1 admin.
2. Prepare 5 sample scanned documents (PDF and image).
3. Create sample signature, stamp and seal images as PNG files with transparent background.
4. Save everything in a shared folder and name files clearly (for example hod_signature.png).

Done when: the sample folder is shared and every file is named clearly.
Needs: Step 2. **Unlocks:** the final project.
Importance: Medium | **Effort:** 3 points (about 3 hours)

PHASE 1 — Foundation (Weeks 2 and 3)

Goal: Users can register, log in and upload a document whose fingerprint is saved. The web login page and app login screen exist.

Phase flow	Details
INPUTS (must exist before this phase)	Database design, API list, UI designs, dummy data and sample files from Phase 0.
STEPS IN ORDER	**Step 8** - Create the database in code *(Shreeyash)* **Step 9** - Registration and login with password hashing *(Shravani)* **Step 10** - Role-based access *(Shravani)* **Step 11** - Document upload with validation *(Shravani)* **Step 12** - SHA-256 fingerprint at upload *(Shreeyash)* **Step 13** - Base layout, login and registration pages *(Shravani)* **Step 14** - Flutter project and login screen *(Shreya Sutar)* **Step 15** - Approver profile API (signature, stamp, seal) *(Shreya Loya)*
PARALLEL WORK	Steps 13, 14 and 15 start once login (Step 9) works. Backend Steps 8 to 12 run in order.
OUTPUTS (you will have)	Database in code; register and login; role checks; secure upload with SHA-256 hash; web and app login; approver profile API.

STEP 8 — **Shreeyash** — **Week 2**

Create the database in code

Why it matters: This turns your drawing into a real database file the whole team can use.
What to do: Turn the table drawing into real Python models.
How, one action at a time:

1. In backend/models.py write four classes (User, Document, ApprovalStep, Timeline) using SQLAlchemy. Each class is one table; each variable is one column.
2. In app.py create the Flask app, connect SQLite, and run `db.create_all()` to create the database file.
3. Open the file with DB Browser for SQLite and check that every table and column exists.

Done when: the database file has the 4 tables with the right columns.
Learn first: SQLAlchemy models, Flask-SQLAlchemy setup.
Needs: Step 4. **Unlocks:** Step 9.
Importance: Critical | **Effort:** 5 points (about 5 hours)

STEP 9 — **Shravani** — **Week 2**

Registration and login with password hashing

Why it matters: Passwords must never be stored as plain text. This is the base of all security.
What to do: Let users create accounts and log in safely.
How, one action at a time:

1. Build `POST /api/register` and `POST /api/login`.
2. Hash passwords with `generate_password_hash` and check them with `check_password_hash` from werkzeug.security.
3. On successful login return a token (JWT) or start a session, together with the user's role.
4. Test everything in Postman before any page exists.

Done when: Postman can register a user and log in and gets a token. A wrong password is rejected.
Learn first: Flask routes, password hashing, sessions versus JWT.
Needs: Step 8. **Unlocks:** Steps 10, 11, 13, 14, 15, 24.
Importance: High | **Effort:** 6 points (about 6 hours)

> **Tip:** Hashing is one-way: you can check a password against a hash but never turn the hash back into the password.

STEP
10
Shravani
Week 2

Role-based access

Why it matters: Without it, an applicant could approve their own document.
What to do: Make sure each role sees and does only what it should.
How, one action at a time:

1. Write a small decorator that checks the logged-in user's role before running a route.
2. Protect routes: admin-only routes reject others; an approver sees only documents waiting for them; an applicant sees only their own documents.

Starter code:

```
from functools import wraps
from flask import jsonify

def requires_role(role):
    def wrapper(fn):
        @wraps(fn)
        def inner(*args, **kwargs):
            if current_user_role() != role:      # read role from token
                return jsonify(error="Forbidden"), 403
            return fn(*args, **kwargs)
        return inner
    return wrapper
```

Done when: calling an approver route with an applicant token returns a clear 403 error.
Learn first: Python decorators (simple level).
Needs: Step 9. **Unlocks:** Steps 27, 30.
Importance: High | **Effort:** 4 points (about 4 hours)

STEP
11
Shravani
Week 3

Document upload with validation

Why it matters: Users can upload anything, so the system must refuse dangerous or oversized files.
What to do: Accept scanned documents safely.
How, one action at a time:

1. Build `POST /api/documents/upload`.
2. Allow only PDF, JPG and PNG, with a size limit (for example 10 MB). Clean the name with `secure_filename` and save it with a unique name in the uploads folder.
3. Create a document row with status in_progress and current_step 1.

Done when: uploading a PDF in Postman saves the file and a row; uploading an .exe file is rejected.
Learn first: Flask file uploads, secure file names, file validation.
Needs: Step 9. **Unlocks:** Step 12.
Importance: High | **Effort:** 5 points (about 5 hours)

STEP
12
Shreeyash
Week 3

SHA-256 fingerprint at upload

Why it matters: This fingerprint is how the system proves later that nobody edited the file.
What to do: Save the file's fingerprint (hash) so tampering can be detected later.
How, one action at a time:

1. Write utils/hashing.py with the function below that reads the file in chunks.
2. Call it right after saving the upload and store the result in documents.original_hash.
3. Write verify_file(path, stored_hash) that returns True if the new hash equals the stored one, else False.
4. Test: upload a file, change one byte, and check verify_file gives False.

Starter code:

```
import hashlib

def sha256_file(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b""):
            h.update(chunk)
    return h.hexdigest()          # 64 characters

def verify_file(path, stored_hash):
    return sha256_file(path) == stored_hash
```

Done when: uploaded documents have a 64-character hash; changing one byte of the file makes verify_file return False.
Learn first: What a hash is, hashlib, reading files in chunks.
Needs: Step 11. **Unlocks:** Steps 16, 21.
Importance: Critical | **Effort:** 5 points (about 5 hours)

> **Tip:** A hash is a fingerprint, not a lock. It does not hide the file; it only proves whether the file is still the same.

STEP
13
Shravani
Week 2

Base layout, login and registration pages

Why it matters: These are the first pages everyone sees, and they set the design for all later pages.
What to do: Build the first web pages.
How, one action at a time:

1. Create a base template (navbar, footer, CSS) following Shreya Sutar's designs.
2. Build login and registration pages with form checks in JavaScript (empty fields, email format, password length).
3. Use `fetch()` to call the login and register endpoints and save the token in the browser.

Done when: a user can register and log in from the browser and is taken to a dashboard placeholder.
Learn first: HTML forms, CSS Flexbox, JavaScript fetch and async/await.
Needs: Steps 5, 6, 9. **Unlocks:** the final project.
Importance: Medium | **Effort:** 5 points (about 5 hours)

STEP
14
Shreya Sutar
Week 2

Flutter project and login screen

Why it matters: Starting the app early reveals setup problems while there is still time to fix them.
What to do: Start the mobile app.
How, one action at a time:

1. Create the Flutter project in the mobile folder and run it on an emulator or a real phone.
2. Build login and register screens from the designs.
3. Use the http package to call the login API and store the token with flutter_secure_storage.

Done when: you can log in from the app and a token is stored.
Learn first: Dart basics, Flutter widgets, http package.
Needs: Steps 5, 6, 9. **Unlocks:** the final project.
Importance: Medium | **Effort:** 6 points (about 6 hours)

STEP 15 Shreya Loya Week 3	**Approver profile API (signature, stamp, seal)** **Why it matters:** The final document needs each approver's own images. This API collects them. **What to do:** Let each approver upload the three images that will be printed on documents. **How, one action at a time:** 1. Build `POST /api/profile/assets` accepting three PNG images: signature, stamp, seal. 2. Validate file type and size, save in uploads/assets/user_id, and store the paths in the users table. 3. Build `GET /api/profile` to return what is uploaded. 4. Agree with Shreeyash on: PNG only, transparent background, fixed folder names. **Done when:** an approver can upload all three images and fetch them again; a non-PNG file is rejected. **Learn first:** Flask file upload and basic validation. **Needs:** Step 9. **Unlocks:** Steps 25, 29. **Importance:** High | **Effort:** 5 points (about 5 hours)

PHASE 2

Core approval workflow (Weeks 4 and 5)

Goal: A document can travel through several approvers with approve, pending and compliance, and every move is recorded and checked.

Phase flow	Details
INPUTS (must exist before this phase)	Working login, roles, secure upload and SHA-256 hashing from Phase 1.
STEPS IN ORDER	**Step 16** - Approver selection and approval chain *(Shreya Loya)* **Step 17** - Multi-approver sequence logic *(Shreeyash)* **Step 18** - Approve, pending and compliance actions *(Shreeyash)* **Step 19** - Compliance flow (take back or fulfil) *(Shreya Loya)* **Step 20** - Live approval timeline *(Shreya Sutar)* **Step 21** - Hash verification before each action *(Shreeyash)* **Step 22** - Applicant and approver dashboards (web) *(Shravani)* **Step 23** - Applicant and approver screens (app) *(Shreya Sutar)* **Step 24** - Search and filter API *(Shravani)*
PARALLEL WORK	Dashboards (22, 23) and search API (24) run alongside the backend flow (16 to 21).
OUTPUTS (you will have)	Approver chain; multi-approver sequence; approve / pending / compliance; compliance flow; live timeline; hash checks; dashboards on web and app; search API.

STEP 16
Shreya Loya
Week 4

Approver selection and approval chain

Why it matters: This is where the document gets its route: who approves, and in what order.
What to do: Create the ordered list of approvers for a document.
How, one action at a time:

1. Build `GET /api/approvers?post=HOD&department=IT` so the uploader can search approvers by post.
2. Build `POST /api/documents/{id}/chain` that receives approver ids in order and creates approval_steps rows with step_order 1, 2, 3.
3. Set step 1 to pending and all the others to waiting.

Done when: a document with 3 approvers shows step 1 pending and steps 2 and 3 waiting in the database.
Learn first: SQLAlchemy relationships and queries, Flask routes.
Needs: Step 12. **Unlocks:** Step 17.
Importance: High | **Effort:** 6 points (about 6 hours)

STEP 17
Shreeyash
Week 4

Multi-approver sequence logic

Why it matters: This is the heart of the project: the document must reach approvers strictly one after another.
What to do: Make the document move from one approver to the next in order.
How, one action at a time:

1. Write a function that finds the current step, checks that the logged-in approver owns it, and refuses everyone else.
2. When the current step is approved, mark it approved, add 1 to current_step and set the next step to pending.
3. When the last step is approved, set the document status to fully_approved.
4. Draw the state diagram (waiting, pending, approved, compliance) before coding.

Starter code:

```
def approve_step(step, doc):
    step.status = "approved"
    next_step = ApprovalStep.query.filter_by(
        document_id=doc.id, step_order=step.step_order + 1).first()
    if next_step:
        next_step.status = "pending"       # next approver now sees it
        doc.current_step = next_step.step_order
    else:
        doc.status = "fully_approved"      # last approver finished
    db.session.commit()
```

Done when: approver 2 cannot see or act on the document until approver 1 approves; after approver 3 approves the status becomes fully_approved.
Learn first: State machine thinking, SQLAlchemy queries.
Needs: Step 16. **Unlocks:** Step 18.
Importance: Critical | **Effort:** 8 points (about 8 hours)

> **Tip:** Draw boxes and arrows on paper first. Most bugs in workflow code come from not knowing every possible state.

STEP 18
Shreeyash
Week 4

Approve, pending and compliance actions

Why it matters: These three choices are what makes the project a real approval system.
What to do: Give approvers their three choices.
How, one action at a time:

1. Build `POST /api/steps/{id}/approve`, `/pending` and `/compliance`. Each accepts an optional remark.
2. Approve: create a unique approval number and move the sequence forward (Step 17 code).
3. Pending: keep the document at the same step and store the remark.
4. Compliance: set the document to compliance_required and store the remark for the applicant.

Starter code:

```
from datetime import datetime

def make_approval_no(doc_id, step_order):
    return f"APH-{datetime.now().year}-{doc_id:05d}-{step_order}"
# example: APH-2026-00012-2
```

Done when: all three actions work in Postman and update the database as expected.
Needs: Step 17. **Unlocks:** Steps 19, 20, 21, 22, 23, 25.
Importance: Critical | **Effort:** 7 points (about 7 hours)

STEP 19
Shreya Loya
Week 5

Compliance flow (take back or fulfil)

Why it matters: Real approvers often ask for corrections. Without this the applicant would have to start over.
What to do: Let the applicant respond when an approver sends compliance.
How, one action at a time:

1. Build `POST /api/documents/{id}/fulfil`: the applicant uploads a corrected file, a new hash is stored, and the same approver gets it again.
2. Build `POST /api/documents/{id}/withdraw` for taking the request back.
3. Keep the old file's hash in the timeline for history.

Done when: after compliance, the applicant can fix and resubmit, and the document returns to the same approver.
Learn first: Flask file upload, SQLAlchemy updates.
Needs: Step 18. **Unlocks:** Steps 22, 23.
Importance: High | **Effort:** 6 points (about 6 hours)

STEP 20
Shreya Sutar
Week 5

Live approval timeline

Why it matters: The applicant must see progress at every moment; this also acts as an audit log.
What to do: Record every event so progress can be followed.
How, one action at a time:

1. Write one helper `add_timeline(document_id, text)` and call it at every important point: uploaded, sent to approver, approved by X, compliance by Y, fulfilled, completed.
2. Build `GET /api/documents/{id}/timeline` returning events in time order.

Done when: after a full approval run the timeline reads like a story from upload to completion.
Learn first: Flask routes, SQLAlchemy queries.
Needs: Step 18. **Unlocks:** Steps 28, 29.
Importance: High | **Effort:** 4 points (about 4 hours)

STEP 21
Shreeyash
Week 5

Hash verification before each action

Why it matters: This is the tamper-detection feature: every action first checks the file is unchanged.
What to do: Detect tampering at every stage.
How, one action at a time:

1. Before any approve, pending or compliance action, run verify_file on the current file and compare with the stored hash.
2. If it fails, block the action, set a tamper flag and add the timeline event 'Tampering detected'.
3. Test by editing a stored file by hand and trying to approve it.

Done when: a modified file is refused and flagged; an untouched file works normally.
Needs: Steps 12, 18. **Unlocks:** the final project.
Importance: Critical | **Effort:** 4 points (about 4 hours)

STEP 22
Shravani
Week 4

Applicant and approver dashboards (web)

Why it matters: These are the pages users actually use every day.
What to do: Build the main working pages.
How, one action at a time:

1. Applicant dashboard: upload form, approver selection by post and department, and a list of my documents with coloured status badges.
2. Approver dashboard: pending list with buttons Approve, Keep Pending, Send Compliance and a remark box.
3. Connect with `fetch()` to the real endpoints. While the backend is not ready, use fake JSON data.

Done when: an applicant can upload and pick approvers, and an approver can see and act on a document, all from the browser.
Learn first: DOM manipulation, rendering lists from JSON, CSS for tables and badges.
Needs: Steps 18, 19. **Unlocks:** Step 31.
Importance: High | **Effort:** 9 points (about 9 hours)

STEP 23
Shreya Sutar
Week 4

Applicant and approver screens (app)

Why it matters: The app is the mobile version of the same journey.
What to do: Build the main app screens.
How, one action at a time:

1. Applicant: upload using camera or gallery (image_picker), approver selection, document list with status.
2. Approver: pending list and action buttons with a remark.
3. Convert JSON from the API into Dart model classes.

Done when: an applicant can upload from the phone and an approver can act on a document in the app.
Learn first: ListView, forms, navigation, JSON parsing, image_picker.
Needs: Steps 18, 19. **Unlocks:** Step 31.
Importance: High | **Effort:** 9 points (about 9 hours)

STEP 24
Shravani
Week 5

Search and filter API

Why it matters: With many documents, users need to find one quickly.
What to do: Let users find documents quickly.
How, one action at a time:

1. Build `GET /api/documents?status=&q=&from=&to=` that filters by status, title keyword and date range.
2. Use SQLAlchemy filters, with LIKE for keyword search.
3. Respect roles: applicants see only their documents, approvers see only theirs.

Done when: searching by keyword and filtering by status gives correct results in Postman.
Learn first: SQL WHERE, LIKE, ORDER BY, SQLAlchemy filtering.
Needs: Step 9. **Unlocks:** Step 28.
Importance: Medium | **Effort:** 4 points (about 4 hours)

PHASE 3

Document processing and extras (Week 6)

Goal: The approved document carries all signatures, stamps, seals and the approval number, and the remaining features are complete.

Phase flow	Details
INPUTS (must exist before this phase)	Working approval flow (Phase 2) and each approver's signature, stamp and seal images.
STEPS IN ORDER	**Step 25** - Stamp signatures, stamps, seals and approval number *(Shreeyash)* **Step 26** - Final document and final hash *(Shreya Sutar)* **Step 27** - PIN lock logic *(Shreya Sutar)* **Step 28** - Status page, search and filter UI, download page (web) *(Shravani)* **Step 29** - Timeline, profile, PIN and biometric lock (app) *(Shreya Sutar)* **Step 30** - Admin panel and download center API *(Shreya Loya)*
PARALLEL WORK	Steps 25, 27 and 30 run in parallel. Pages 28 and 29 start after the timeline and download endpoints exist.
OUTPUTS (you will have)	Stamped final PDF with final hash; PIN lock; web status, search and download pages; app timeline, profile and biometric lock; admin panel.

STEP 25
Shreeyash
Week 6

Stamp signatures, stamps, seals and approval number

Why it matters: This produces the final approved document, the visible result of the project.
What to do: Produce the final approved document.
How, one action at a time:

1. Fix positions in advance: for example approver 1 bottom-left, approver 2 bottom-centre, approver 3 bottom-right, with the approval number under each block.
2. If the upload is a JPG or PNG, convert it to a PDF first with Pillow.
3. With PyMuPDF open the PDF and call insert_image at the chosen position for each approver's signature, stamp and seal, then insert_text for the approval number.
4. Always save as a new file. Never overwrite the original.

Starter code:

```
import fitz   # PyMuPDF

pdf = fitz.open("original.pdf")
page = pdf[-1]                              # last page
rect = fitz.Rect(50, 700, 150, 760)         # x0, y0, x1, y1 (points)
page.insert_image(rect, filename="hod_signature.png")
page.insert_text((50, 775), "APH-2026-00012-1", fontsize=9)
pdf.save("final.pdf")
```

Done when: a 3-approver test document comes out with all three signature blocks and approval numbers in the right places.
Learn first: PyMuPDF (fitz), PDF coordinate system, Pillow image resize.
Needs: Steps 15, 18. **Unlocks:** Step 26.
Importance: Critical | **Effort:** 9 points (about 9 hours)

> **Tip:** PDF coordinates start at the top-left corner and are measured in points. Try on one test PDF and adjust the numbers until it looks right.

STEP 26
Shreya Sutar
Week 6

Final document and final hash

Why it matters: Locking the finished document means nobody can change it after approval without detection.
What to do: Lock the finished document.
How, one action at a time:
1. Compute SHA-256 of the stamped file and store it in documents.final_hash and final_path.
2. Build `GET /api/documents/{id}/download` that re-checks the final hash before sending the file.
3. Add the timeline event 'Final document generated'.

Done when: downloading a final document works; a modified final file is refused.
Learn first: Flask send_file, reusing the hashing function from Step 12.
Needs: Step 25. **Unlocks:** Steps 28, 31.
Importance: High | **Effort:** 4 points (about 4 hours)

STEP 27
Shreya Sutar
Week 6

PIN lock logic

Why it matters: A PIN adds a second layer of protection on web and app.
What to do: Support the extra PIN protection.
How, one action at a time:
1. Build `POST /api/pin/set` and `POST /api/pin/verify`. Store only the hashed PIN (same hashing as passwords).
2. Limit wrong attempts (for example 5) and then block for a few minutes.

Done when: a PIN can be set and verified, and 5 wrong tries lock it for a few minutes.
Learn first: Hashing a PIN (same method as passwords), Flask routes.
Needs: Step 10. **Unlocks:** Step 29.
Importance: High | **Effort:** 4 points (about 4 hours)

STEP 28
Shravani
Week 6

Status page, search and filter UI, download page (web)

Why it matters: This completes the website and makes the timeline visible.
What to do: Complete the website.
How, one action at a time:
1. Status page: show the live timeline for one document.
2. Search box and filter dropdowns that call the search API.
3. Download center page listing final documents with download buttons.
4. PIN entry page that calls the PIN API.

Done when: every page in the designs exists and works on desktop and on a phone-sized screen.
Learn first: Responsive CSS and media queries.
Needs: Steps 20, 24, 26. **Unlocks:** Step 31.
Importance: Medium | **Effort:** 7 points (about 7 hours)

STEP 29
Shreya Sutar
Week 6

Timeline, profile, PIN and biometric lock (app)

Why it matters: These screens complete the app and add its special mobile features.
What to do: Complete the app.
How, one action at a time:
1. Timeline screen showing events from the API.
2. Profile screen where an approver uploads signature, stamp and seal with the profile API.
3. PIN screen and fingerprint or face lock using the local_auth package.
4. If Flutter becomes too hard, make the website fully mobile-responsive as a fallback.

Done when: the app has all screens from the designs and the biometric prompt appears when opening the app.
Learn first: local_auth package, file picker, Flutter navigation.
Needs: Steps 15, 20, 27. **Unlocks:** Step 31.
Importance: Medium | **Effort:** 8 points (about 8 hours)

STEP 30 Shreya Loya Week 6	**Admin panel and download center API** **Why it matters:** The admin creates approvers, and users need to fetch their final documents. **What to do:** Give the admin control and finish the remaining APIs. **How, one action at a time:** 1. Admin routes: list users, add or edit a user, set role, post and department, deactivate a user. 2. Download center API: list the final documents of the logged-in user. 3. A simple admin web page for these actions (coordinate with Shravani for styling). **Done when:** the admin can create an approver with a post and department, and that approver appears in the approver search. **Learn first:** Admin-only routes, CRUD operations. **Needs:** Step 10. **Unlocks:** Step 31. **Importance:** High | **Effort:** 7 points (about 7 hours)

PHASE 4 — Integration, testing and deployment (Week 7)

Goal: Everything works together, is tested from start to end, and runs online.

Phase flow	Details
INPUTS (must exist before this phase)	All modules from Phases 1 to 3 finished by their owners.
STEPS IN ORDER	**Step 31** - Integrate web and app with the real API *(Shreeyash)* **Step 32** - Full testing and bug list *(Shreya Loya)* **Step 33** - Fix bugs and run a security check *(Everyone)* **Step 34** - Deploy online *(Shravani)*
PARALLEL WORK	Strictly in order: 31, 32, 33, 34.
OUTPUTS (you will have)	Integrated web, app and API; test results and bug list; fixed bugs; live deployed project.

STEP 31 — Shreeyash — Week 7

Integrate web and app with the real API

Why it matters: Parts built separately often do not fit; this step makes them fit.
What to do: Replace all fake data with real API calls and fix mismatches.
How, one action at a time:

1. Merge all branches into main with Git and resolve conflicts together.
2. Sit with the other three members and run each page and screen against the real backend.
3. Fix field name or format mismatches either in the API or in the frontend.

Done when: the complete journey works on both web and app without fake data.
Needs: Steps 22, 23, 26, 28, 29, 30. **Unlocks:** Step 32.
Importance: Critical | **Effort:** 5 points (about 5 hours)

STEP 32 — Shreya Loya — Week 7

Full testing and bug list

Why it matters: Bugs found now cost minutes; bugs found during the demo cost marks.
What to do: Find problems before the demo.
How, one action at a time:

1. Write at least 40 test cases in a sheet: test id, steps, expected result, actual result, status, severity.
2. Test: single approver, three approvers in order, wrong approver trying to act, compliance and fulfil, withdraw, tampered file, wrong password, wrong file type, large file, PIN lock.
3. Report each bug to its owner with exact steps to reproduce.

Done when: every test case has a result and every serious bug has an owner.
Learn first: Writing test cases, bug reports.
Needs: Step 31. **Unlocks:** Step 33.
Importance: High | **Effort:** 8 points (about 8 hours)

STEP 33 — Everyone — Week 7

Fix bugs and run a security check

Why it matters: A working project must also be a safe one.
What to do: Make the project reliable and secure.
How, one action at a time:

1. Each owner fixes bugs in their own module.
2. Check: passwords are hashed, routes are protected by role, file types are checked, no secret keys are pushed to GitHub, uploads folder is not directly browsable.

Done when: no serious bug remains in the bug list.
Needs: Step 32. **Unlocks:** Step 34.

STEP 34 Shravani Week 7	**Deploy online** **Why it matters:** A live link lets the guide and examiners try the project on their own phones. **What to do:** Put the backend online. **How, one action at a time:** 1. Use Render or PythonAnywhere. Add gunicorn and set environment variables for secret keys. 2. Update the app and web to use the live URL. 3. Test on a real phone with mobile data. **Done when:** you can open the live link from a phone and complete one full approval. **Learn first:** Environment variables, gunicorn, deploying a Flask app. **Needs:** Step 33. **Unlocks:** Steps 35, 36. **Importance:** High | **Effort:** 5 points (about 5 hours)

PHASE 5 — Documentation and submission (Week 8)

Goal: Report, presentation and demo are ready.

Phase flow	Details
INPUTS (must exist before this phase)	A deployed and tested project from Phase 4.
STEPS IN ORDER	**Step 35** - README and API documentation *(Shreya Sutar)* **Step 36** - Final report, literature survey and references *(Shreya Loya)* **Step 37** - Presentation and demo script *(Shreya Loya)* **Step 38** - Final review and submission *(Everyone)*
PARALLEL WORK	Steps 35, 36 and 37 run in parallel. Step 38 is last.
OUTPUTS (you will have)	README and API docs; final report; PPT and demo script; submitted project.

STEP 35
Shreya Sutar
Week 8

README and API documentation

Why it matters: Good documentation shows professionalism and helps you in interviews.
What to do: Document how to run and use the project.
How, one action at a time:

1. Write a README with project description, features, tech stack, setup steps and screenshots.
2. Turn the API list into final documentation with example requests and responses.

Done when: someone new can follow the README and run the project.
Needs: Step 34. **Unlocks:** Step 38.
Importance: Medium | **Effort:** 4 points (about 4 hours)

STEP 36
Shreya Loya
Week 8

Final report, literature survey and references

Why it matters: The report is what the college officially evaluates.
What to do: Write the college submission.
How, one action at a time:

1. Follow the 8 headings: title, objectives, literature survey, work methodology, features/hardware/software/cost estimation, expected outcome, conclusion and future scope, references.
2. Collect a short paragraph from each member about their own module.
3. Add screenshots, the database diagram and the flow diagram.

Done when: the report is complete and the guide has reviewed it.
Needs: Step 34. **Unlocks:** Steps 37, 38.
Importance: Medium | **Effort:** 8 points (about 8 hours)

STEP 37
Shreya Loya
Week 8

Presentation and demo script

Why it matters: A good demo makes the project look as strong as it is.
What to do: Prepare to present.
How, one action at a time:

1. Make 10 to 12 slides: problem, solution, features, architecture, database, screens, demo, future scope.
2. Write a 5-minute demo script: upload, choose 3 approvers, approve in order, compliance case, tamper detection, final stamped document.
3. Everyone rehearses their own part.

Done when: the team can run the full demo in 5 minutes without errors.
Needs: Step 36. **Unlocks:** Step 38.
Importance: Medium | **Effort:** 5 points (about 5 hours)

STEP 38 Everyone Week 8	**Final review and submission** **Why it matters:** Closing the project properly. **What to do:** Close the project. **How, one action at a time:** 1. Show everything to the guide, note the feedback and fix it. 2. Tag the final GitHub version and submit the report. **Done when:** report submitted and demo approved by the guide. **Needs:** Steps 35, 36, 37. **Unlocks:** the final project.

8. Learning flow

Learn each topic just before the step that needs it. Learn, build a small version, then build the real one.

Week	Everyone	Shreeyash	Shravani	Shreya Sutar	Shreya Loya
1	Git and GitHub, HTTP, JSON, REST, Postman	Python basics, SQL tables and keys, project structure	Python and Flask basics, HTML and CSS	UI/UX basics, Figma, Dart basics	Python and Flask basics, REST and API design basics
2	Read the API list	SQLAlchemy models	Password hashing, JWT or sessions, JavaScript fetch	Flutter widgets, forms, http package	SQLAlchemy relationships, Flask file upload
3	-	hashlib and SHA-256, reading files in chunks	File upload and validation, responsive CSS	JSON parsing, Provider basics	Profile upload practice, writing test cases
4-5	-	State machines, workflow logic	Rendering lists from JSON, SQL filtering and search	ListView, image_picker, Flask routes for timeline	Compliance flow logic, SQLAlchemy queries, Postman collections
6	-	PyMuPDF, Pillow, PDF coordinates	Media queries, polishing pages	local_auth, secure storage, PIN hashing, file download	Admin CRUD, download API
7	Debugging, reading error messages	Git merging, integration	Deployment: Render, gunicorn, environment variables	Building the APK	Full testing and bug reports
8	Presentation and demo skills	Review and support	Screenshots for the report	README writing	Report, PPT, references

Why this order: each week's topics match the steps of that week (Section 7). If you are ahead, read next week's topics. If you are behind, ask for help early.

9. Tools, topics and technologies in detail

Detailed tables for each person: what to learn, why it is needed, and a practice task. Use them together with the week-by-week plan in Section 8.

Free resources: official documentation (Flask, Flutter, Python, MDN for web), freeCodeCamp, CS50, W3Schools; YouTube channels Corey Schafer (Python and Flask), Traversy Media (web) and The Net Ninja (Flutter).

A. Everyone

Topic / tool	What to learn	Why needed	Practice task
Git and GitHub	init, add, commit, branch, merge, pull, push, resolving conflicts, .gitignore	Four people share one codebase	Create the repo, make a branch each, merge a change
VS Code	Extensions, terminal, debugging, formatting	Main code editor	Set up the Python, Flutter and Live Server extensions
How the web works	Client and server, HTTP methods (GET, POST), status codes (200, 401, 403, 404), JSON, REST	The project is clients calling an API	Open DevTools Network tab and watch requests
Postman	Sending requests, headers, body, tokens, saving collections	Test the API before the UI exists	Test a public API, then your own
Teamwork basics	Task boards, code review, clear commit messages	Avoid conflicts and duplicate work	Use Trello or GitHub Projects for your tasks

B. Shreeyash - core engine and integration

Topic / tool	What to learn	Why needed	Practice task
Python core	Variables, loops, functions, lists, dictionaries, classes, file handling, exceptions, modules, virtual environments, pip	Foundation for everything	File organizer or to-do program
SQL and SQLite	Tables, primary and foreign keys, one-to-many, SELECT, INSERT, UPDATE, DELETE, JOIN	Stores users, documents, steps, timeline	Library database with 3 tables and queries
SQLAlchemy ORM	Models, relationships, sessions, queries, Flask-Migrate basics	Python code instead of raw SQL	Rebuild the library database as models
SHA-256 and hashlib	What a hash is, why one changed byte changes everything, chunked file reading, comparing hashes, hashing versus encryption	Tamper detection	Hash a file, edit it, re-hash and compare
Workflow and state machines	States, transitions, ordering, who can act when	Multi-approver sequence and actions	Draw the flow on paper, then code it
Approval numbers	Unique IDs, formats like APH-2026-00012-1, uniqueness checks	Every approval needs a number	Write a generator that never repeats
PDF and image processing	PyMuPDF: open PDF, insert image at coordinates, insert text, page size. Pillow: open, resize, transparency. Coordinate system	Stamping sign, stamp, seal, number	Stamp a logo on a PDF at a fixed position
Git workflow and code review	Branching, merging, resolving conflicts, reviewing teammates' changes	Integration and quality	Merge two branches with a conflict and fix it

Topic / tool	What to learn	Why needed	Practice task
Integration and debugging	Reading error messages, logging, testing endpoints with Postman, matching frontend and API	Joining all modules	Trace one request from page to database

C. Shravani - access, web and deployment

Topic / tool	What to learn	Why needed	Practice task
Python and Flask basics	Routes, request and response, JSON responses, config, error handlers, CORS	Builds her APIs	Notes API with add, list, delete
Authentication and security	Password hashing, sessions versus JWT, safe error messages, basic OWASP risks (SQL injection, XSS, broken access control)	Login system	Login system with 3 roles and protected routes
Role-based access	Python decorators (simple level), checking a role before a route runs	Each role sees only its own pages	Protect an admin-only route
File upload and validation	Flask file uploads, secure file names, allowed types and size limits, unique file names	Document upload API	Upload endpoint that rejects wrong file types
SQL filtering	SELECT with WHERE, LIKE, ORDER BY, SQLAlchemy filters	Search and filter API	Search users by name and department
HTML5 and CSS3	Forms, tables, semantic tags, Flexbox, Grid, media queries, responsive design	Website structure and layout	Clone a simple dashboard layout
JavaScript	Variables, functions, DOM, events, fetch API, async/await, JSON, form validation, localStorage	Makes pages talk to the API	Fetch and display a list from a public API
Jinja templates and DevTools	Template inheritance, loops, passing data from Flask; Inspect, console, network tab	Pages served by Flask, debugging	Base layout with two child pages
Deployment	Environment variables, requirements.txt, gunicorn, Render or PythonAnywhere, development versus production	Puts the project online	Deploy a hello-world Flask app

D. Shreya Sutar - design, mobile app and app APIs

Topic / tool	What to learn	Why needed	Practice task
UI/UX basics	User flows, wireframes, colour theory, typography, consistency, accessibility, mobile-first design	Clear and attractive screens	Wireframe the upload and approval flow
Figma or Canva	Frames, components, auto layout, prototypes, exporting assets	Design deliverable	Design login and dashboard screens
Dart	Variables, functions, classes, null safety, async/await, Futures, collections	Language of Flutter	Small console programs
Flutter basics	Widgets, Stateless versus Stateful, Row, Column, Stack, ListView, forms, navigation, themes	Builds every screen	A 3-screen sample app
State management	setState first, then Provider	Share user and document data across screens	Counter app, then a list app with Provider
Networking in Flutter	http package, JSON parsing, model classes, error handling, loading states, flutter_secure_storage	Connects the app to the API	Fetch and show data from a public API

Topic / tool	What to learn	Why needed	Practice task
Device features	image_picker (camera, gallery), file_picker, local_auth (fingerprint, face), permissions	Scan and upload, biometric lock	Pick an image and show it; fingerprint prompt
Small Flask APIs	Routes, SQLAlchemy queries, sending a file with send_file, hashing a PIN	Timeline, PIN and final download APIs	Build a PIN set and verify endpoint
Documentation	README, setup steps, screenshots, clear writing	Project README	Write a README for a practice project

E. Shreya Loya - workflow support, admin and quality

Topic / tool	What to learn	Why needed	Practice task
Python and Flask basics	Core Python, routes, JSON, simple CRUD	Builds her APIs	A profile image upload endpoint
REST and API design	Methods, URLs, request and response fields, error codes, writing an API list	API list and documentation	Write the API list for a small to-do app
SQLAlchemy relationships	One-to-many, queries across tables, updates	Approver chain and compliance flow	Model a user with many documents
File upload	Saving images, validating PNG, folder structure	Approver profile API	Upload and fetch an image through an API
Admin panel concepts	Admin-only routes, listing, editing, deleting records safely	Manage users, posts, departments	List and edit users on a simple page
Software testing	Test cases (input, steps, expected, actual), positive and negative tests, edge cases, bug reports with severity	Finds problems before the demo	Write 20 test cases for the login module
Postman collections	Saved requests, environments, basic automated tests	Repeatable API testing	Save a collection for the whole API
Technical writing	Synopsis format (8 headings), literature survey, IEEE-style references, report structure	College submission	Literature survey of 5 similar systems
Presentation skills	PPT structure, screenshots, demo scripting, answering questions	Final demo	Rehearse a 5-minute demo

Suggested learning rule

- Learn, build a tiny version, then build the real one.
- Spend 30 minutes reading documentation after every build session and note new things in a shared document.
- If stuck for more than 45 minutes, ask a teammate, then your guide, then Stack Overflow or the official documentation.

10. API list and tools to install

Use this as the base for Step 5 and Step 1.

Method and URL	What it does	Built by (step)
POST /api/register	Create an account	Shravani (9)
POST /api/login	Log in, returns token and role	Shravani (9)
POST /api/pin/set, /api/pin/verify	Set and check the PIN	Shreya Sutar (27)
GET /api/profile	Show my profile and uploaded images	Shreya Loya (15)
POST /api/profile/assets	Upload signature, stamp, seal (PNG)	Shreya Loya (15)
GET /api/approvers?post=&department=	Search approvers by post and department	Shreya Loya (16)
POST /api/documents/upload	Upload a document, store its SHA-256 hash	Shravani (11) with Shreeyash (12)
POST /api/documents/{id}/chain	Choose approvers in order	Shreya Loya (16)
GET /api/documents?status=&q=	List, search and filter my documents	Shravani (24)
GET /api/approvals/pending	Approver's list of documents waiting for them	Shreeyash (17)
POST /api/steps/{id}/approve, /pending, /compliance	Approver actions with an optional remark	Shreeyash (18)
POST /api/documents/{id}/fulfil, /withdraw	Applicant answers compliance or takes the request back	Shreya Loya (19)
GET /api/documents/{id}/timeline	Live progress events	Shreya Sutar (20)
GET /api/documents/{id}/download	Download the final stamped document (hash re-checked)	Shreya Sutar (26)
GET /api/downloads	List my final documents	Shreya Loya (30)
GET and POST /api/admin/users	Admin: list, create and edit users, posts and departments	Shreya Loya (30)

Tools to install

Tool	Used for	Who installs	Quick check
Python 3.11+	Backend code	Everyone	python --version
VS Code	Code editor	Everyone	Open a folder and run a file
Git and GitHub	Share code and track changes	Everyone	git --version
Postman	Test the API without a screen	Everyone	Send a request to a public API
DB Browser for SQLite	Look inside the database file	Shreeyash, Shravani, Shreya Loya	Open the .db file
Figma or Canva	UI designs	Shreya Sutar	Create a frame
Flutter SDK and Android Studio	Mobile app	Shreya Sutar	flutter doctor
draw.io	Database and flow diagrams	Shreeyash, Shreya Loya	Draw 2 boxes
Render or PythonAnywhere	Online deployment	Shravani	Deploy a hello-world app
Excel or Google Sheets	Test cases, bug list, task tracker	Shreya Loya	Create a test case sheet

11. Weekly plan, team rules and final checklist

Week	Must be finished by the end of the week	Steps
1	Tools installed, repository created, database design and API list shared, UI designs ready, dummy data ready	1, 2, 3, 4, 5, 6, 7
2	Database in code, register and login, roles, web login pages, app login screen	8, 9, 10, 13, 14
3	Upload with validation, SHA-256 at upload, approver profile API	11, 12, 15
4	Approver chain, sequence logic, approve and pending and compliance actions, dashboards started	16, 17, 18, 22, 23
5	Compliance flow, timeline, hash verification, dashboards finished, search API	19, 20, 21, 24
6	Stamping, final hash, PIN logic, remaining web and app screens, admin panel	25, 26, 27, 28, 29, 30
7	Integration, testing, bug fixes, deployment	31, 32, 33, 34
8	README, report, PPT, demo rehearsal, submission	35, 36, 37, 38

Team rules

- One repository, one branch per person, merge into main only after checking it works.
- Never change a database field name or an API response without telling the owner and the people who use it.
- Commit small and often, with clear messages like 'Add login API' instead of 'update'.
- Never upload passwords, secret keys or real personal documents to GitHub.
- Keep a shared task sheet. Update it after each work session.
- Test your own module before saying it is done.

Common mistakes to avoid

Mistake	What to do instead
Starting to code without a database design	Finish Step 4 first and share it
Building every feature at once	Build the core flow first, add extras later
Copying code without understanding it	Type it, run it, change it, and explain it to a teammate
Everyone working on main branch	Each person works on their own branch
Waiting silently when blocked	Ask in the group after 45 minutes
Testing only at the end	Test every step with Postman or sample data as soon as it is built
Leaving the report for the last day	Each member writes their module section while building it

Final submission checklist

Done	Item
[ ]	Full flow works: upload, choose approvers, approve in order, compliance, final stamped document
[ ]	Tamper detection demonstrated with an edited file
[ ]	Web and app both connected to the live backend
[ ]	All 40+ test cases have results; no serious bug left
[ ]	Report follows the 8 college headings and is checked by the guide
[ ]	Synopsis first page signed by the guide
[ ]	README, database diagram, flow diagram and screenshots included

Done	Item
[ ]	PPT ready and 5-minute demo rehearsed by all four members
[ ]	Final code tagged and saved on GitHub